

155. Min Stack

Solved

Medium Topics Companies Hint

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int value)` pushes the element `value` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with $O(1)$ time complexity for each function.

two stack , one normal , one mini , in the mini always push the min of value or mini.top()
when pop , pop both

EFFICIENT SOLUTION (USING HELPER STACK)

push(5)

push(-2)

push(0)

push(-3)

getMin() = -3

pop()

top()

getMin()

0

-2

5

-2

5

push:

pop:

getMin:

- push to actual stack

- push to minStack if top is larger

- pop from actual stack

- pop from minStack if top is same

- look at top of minStack

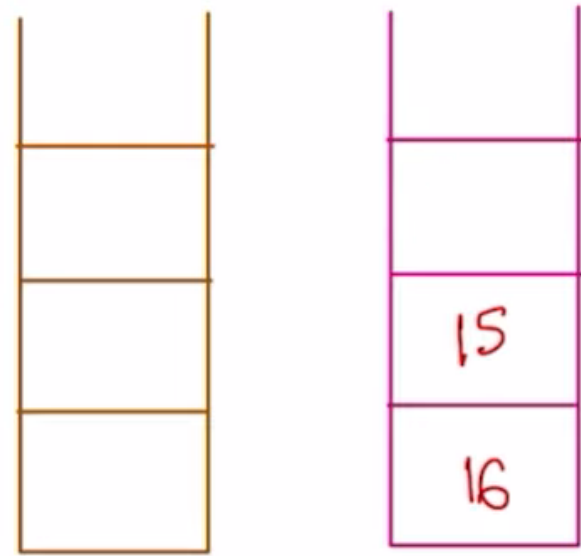


PUSH :

- Just push the element in input stack

POP :

- If output stack is empty
 - Pop all elements from input stack and push in output stack
- Else just pop from output stack



input

output

STACK USING QUEUE

METHOD 1 : USING 2 QUEUES

push (4) --> push (8) --> push (15) --> push (16)

pop () --> pop ()



PUSH :

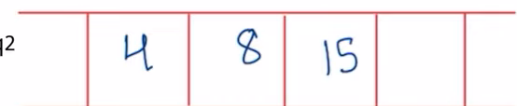
- Just enqueue the elements in q1



q2

POP :

- Count elements in q1 (n)
- For (n - 1) times do this:
 - Dequeue from q1 and enqueue to q2
- Dequeue from q1 and print
- Swap q1 and q2



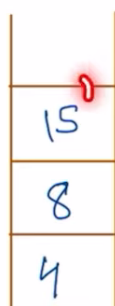
q1



METHOD 2: SINGLE QUEUE

push (4) --> push (8) --> push (15) --> push (16)

pop () --> pop ()



q1

PUSH :

- Add the element to q1
- If the size of queue is 1, don't do anything
- If queue size is greater than 1, let us say (size = n)
- For (n-1) times:
 - Dequeue from q1 and enqueue it back to q1

POP :

- Just dequeue from q1



